

DOSSIER D'ARCHITECTURE LOGICIELLE

Modèle de Sécurité IAM - Rôles Composites & RBAC (Keycloak)

1. Problématique : La Disparition de la notion de "Permission"

Dans les architectures monolithiques classiques (comme la V1 d'IncidentFlow), la sécurité est modélisée via deux entités distinctes : les **Rôles** (profils métiers) et les **Permissions** (droits d'actions techniques), reliés par une table de jointure.

Lors de la migration vers Keycloak et le standard **OAuth2 / OIDC**, cette dichotomie disparaît visuellement. Tout est unifié sous le concept unique de *"Role"*. Pourquoi ? Parce que le standard JWT (JSON Web Token) n'encapsule les droits d'accès que sous la forme d'un unique tableau plat de revendications (*claims*).

2. L'Architecture RBAC via les Rôles Composites

Pour recréer la matrice Rôles/Permissions sans utiliser un moteur UMA complexe, l'architecture V2 s'appuie sur le *Design Pattern* des **Rôles Composites** (Composite Roles) natif à Keycloak.



3. Typologie des Rôles Keycloak

- **Realm Roles (Rôles de Royaume)** : Représentent l'identité métier globale de l'utilisateur au sein de l'entreprise (ex: *Administrateur*, *Employé*, *Technicien*). Ce sont les rôles assignés directement aux humains.
- **Client Roles (Rôles de Client)** : Représentent les permissions techniques (ex: *INCIDENT_CREATE*). Ils sont confinés à une application spécifique (ici le client `incidentflow-frontend`).

En cochant la case "*Composite Roles*" dans Keycloak, le Realm Role devient un simple "dossier" conteneur qui agrège de multiples Client Roles.

4. Avantages Techniques et Héritage

Cette approche surpasse techniquement l'architecture V1 locale :

Elle permet l'**Héritage multiple**. Dans Keycloak, le rôle *Administrateur* peut hériter du rôle *Technicien*, lui-même héritant du rôle *Employé*. Lors de la connexion, Keycloak aplatit automatiquement cet arbre d'héritage et injecte toutes les permissions (Client Roles) sous-jacentes dans le Token JWT, évitant ainsi de lourdes requêtes SQL récursives côté Backend Java.

"Le Pattern des Rôles Composites permet d'unifier la gestion des habilitations. Les administrateurs gèrent des profils métiers (Realm Roles) de haut niveau, pendant que le Backend et le Frontend vérifient des permissions granulaires (Client Roles), garantissant un parfait découplage entre les besoins métiers et l'implémentation technique."